

Week 1: Intros and Web JavaScript

Written by Ben Yan, from past course materials by Jerry Cain

Welcome! 🙌

Hi, I'm Ben, and I'm so excited that you've enrolled in, or thinking about enrolling in CS106S! I hope your first week of autumn quarter is off to a beautiful start, and I look forward to getting to know you! Please feel free to reach out to me at any point! ☺ Happy to answer questions in class, or over email at either bbyan@stanford.edu or judebenlingyan@stanford.edu, and happy to chat after class.

↳ Got a little too invested in the World Cup oops...

HTML, CSS, JavaScript

Modern web applications rely on a triangle of programming technologies: HTML, CSS, and JavaScript.

- **HTML** is the bones and skeleton of a webpage, defining its basic content and structure hierarchy.
- **CSS** is used to regulate color, appearance, and formatting, i.e. making the HTML fashionable.
- **JavaScript (JS)** makes the components above truly interactive, enabling it to operate like a fully-fledged desktop application that just happens to run in a browser. Over 90% of websites use **JS**!

There's a server that delivers all this web content to the browser, and it's often implemented in **Python**!

HTML File

HTML is short for **Hypertext Markup Language**, and you've likely heard of it if you're chronically online like me, or spent any time in school computer lab vandalizing websites with the *Inspect Element* tool ☺.

An **HTML file consists of the raw text to be displayed on the webpage**, interspersed with various formatting directives that are enclosed in angle brackets, called HTML tags, e.g., `<body>`, `<title>`.

HTML tags generally occur in pairs #twinning. The opening tag begins with the name of the tag. The corresponding closing tag has the same name, but preceded with a slash, e.g., `</body>`, `</title>`. The effect of the tag applies to everything between the opening and closing tags. For instance,

`<title> 🦋 CS106S Autumn 2026 🦋 </title>`

Here's a template of what a standard `index.html` file looks like. **This is the naming convention for the homepage of a website**. For instance, when you search up www.cs106s.stanford.edu, you're actually transported to the webpage at the URL www.cs106s.stanford.edu/index.html.

```

<!DOCTYPE HTML>
<html>
  <head>
    <title> title of the page </title>
    Script tags to load JavaScript code
  </head>
  <body>
    Contents of the page, if any.
  </body>
</html>

```



This template features the only noteworthy HTML tags for now. Here are standard HTML principles:

- Every file begins with a `<!DOCTYPE html>` tag. This is their way of saying, *“Hello world, I’m an HTML file! ☺”*, just like me saying, especially often this week,
- The entire content is enclosed in `<html>` and `</html>` tags.
- The file begins with a `<head>` section that specifies the **title** (which appears in the browser tab, not on the page), **JavaScript files to load**, and other details not directly visible on the page itself.
- The file includes a `<body>` section that specifies what’s physically on the page.

Hello world, my name is Ben!



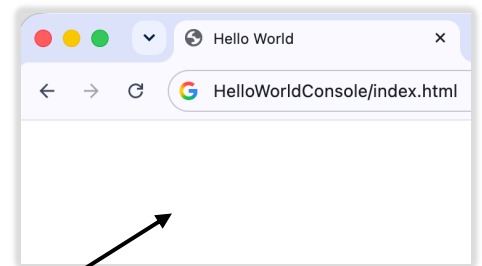
One way of involving JavaScript code in an HTML file is by executing a function (from our script tags) once the HTML content is fully loaded onto the page. To do this, we can revise the `<body>` tag as

```
<body onload="function()"> </body>
```

JavaScript in the Browser

Here’s a compact HTML file that loads JavaScript code from a file called `HelloWorld.js`.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Hello World</title>
    <script text="text/javascript"
      src="HelloWorld.js"></script>
  </head>
  <body onload="HelloWorld()"> </body>
</html>
```



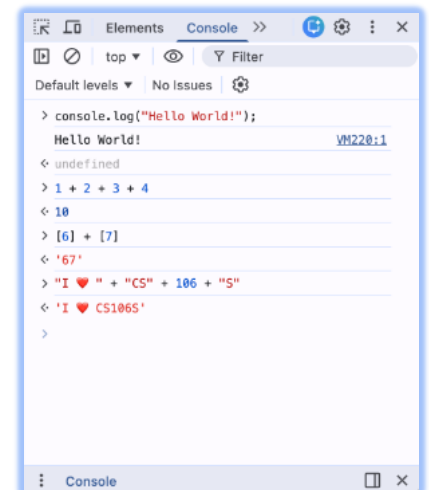
The body of the webpage is initially empty. But once the HTML is loaded, the file asks the browser, through `onload`, to call the function `HelloWorld`. What does this function do? Glad you asked ☺.

```
function HelloWorld() {
  console.log("Hello World!"); // inspired by the first ever C program
}
```

This displays the text `"Hello World!"` on the console log, which appears in different places in different browsers, and can be a bit of an odyssey to find. For Chrome at least, you can right-click the page, and go to **Inspect** (like the infamous Inspect Element tool).

This opens the Chrome Developer Tools, and a panel of tabs should appear reading **Elements**, **Console**, **Sources**, **Network**, **Performance** etc., and you’ll want to click **Console**, where you can interactively type and run JavaScript code, e.g., like on the right.

Notice the **Hello World!** printed, and ta-da, we’re done! 🎉

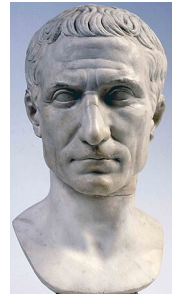


Week 1: JavaScript Practice – Cryptography!

Written by Ben Yan, from past course materials by Jerry Cain

A Caesar cipher, unlike a Caesar salad or Little Caesar's pizza, may actually have been created by Roman general Julius Caesar, who was known to have used the cipher in private correspondence.

The core idea is surprisingly simple: **each letter in the plaintext**—the legible message that we're trying to disguise—**is replaced by a letter a fixed number of positions along the alphabet**. For instance, with a shift of **+3**, "a" would be replaced by "d", "b" would be replaced by "e", and so on, with "w" replaced by "z". **Letters at the end of the alphabet wrap-around**, e.g., "x" would be replaced by "a", and "y" by "b".



Our goal is to follow the JavaScript Tutorial at cs106s.stanford.edu, and ultimately implement **encryptCaesar**, which accepts a plaintext string of lowercase letters and a shift value as input, and **returns the encrypted string that results from shifting each of those letters**. Note the starter code on the course website is decomposed into a few helper functions to complete first!

As optional extensions, you can handle uppercase letters by converting them to lowercase first, then encrypting that individual letter, then translating back to uppercase. You can check if a character is an uppercase letter with `"A" <= ch <= "Z"`, and is a lowercase letter with `"a" <= ch <= "z"`. You can also optionally handle non-letters, e.g., punctuation and spacing, by leaving them as-is in the output.

Here's a handful of examples, with famous last words from Shakespeare's *Julius Caesar* play included.

- `encryptCaesar("abcd", 1)` returns `"bcde"`
- `encryptCaesar("pizza", 3)` returns `"slccd"`
- If you implement the lowercase and non-letter-handling extensions, then `encryptCaesar("Et tu, brute?", 25)` returns `"Ds sg, aqtsd?"`



A Caesar cipher can be drawn as a **Little Caesar's pizza**, with a warm outer crust of letters, and an inner rotating circle of letters. Both run alphabetically clockwise. To encode a letter, you find it on the outer crust, and map it inward to its corresponding letter on the inner circle. The below is for a shift of **+3**.

